

Lecture Note, 1 — Introduction to C
This is a self-made programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by Dennis Ritchie in the late seventies, C began to replace the more familiar languages of that time like PL/I, ALGOL, etc.
Course Code: 25B11C012 • Semester I • Department of Computer Science & Engineering
Instructor/Dept Resource: HJT, Solan Academic Faculty

ANSI C standard emerged in the early 1980s; this book/standard was split into two titles: The original was still called Programming in C, and the title that covered ANSI C was called Programming in ANSI C. This was done because it took several years for compiler vendors to release their ANSI C compilers and for them to become ubiquitous.

Major parts of popular operating systems like Windows, UNIX, and Linux are still written in C. This is because even today, when it comes to performance (speed of execution), nothing beats C. Moreover, if one is to extend the operating system to work with new devices, one needs to write device driver programs. These programs are exclusively written in C. C seems so popular because it is reliable, simple and easy to use.

3. Steps in Learning C vs English Language

- Learning English: Alphabets → Words → Sentences → Paragraphs
- Learning C: Alphabets/Digits/Special Symbols → Constants, Variables, Keywords → Instructions → Program

In machine level language, the computer only understands digital numbers i.e. in the form of 0 and 1. Instructions given to the computer in binary digit form are difficult to maintain and error prone.

Low-Level vs High-Level Languages & Toolchains

1. Machine Level & Assembly Language

The assembly language is a modified version of machine level language, where instructions are given in English-like mnemonics such as ADD, SUM, MOV etc. The translator used here is an Assembler.

2. High Level Language & Translators

High-level languages (Pascal, Fortran, C, C++) are machine independent (portable).

Three fundamental types of translators:

- Compiler: Translates the whole high-level source code into machine language at once (Object Program).
- Interpreter: Translates source code line-by-line; stops and reports upon finding an error.
- Assembler: Translates assembly mnemonics into machine code.

3. Integrated Development Environments (IDEs)

The process of editing, compiling, running, and debugging programs is managed by an IDE.

Examples: Microsoft Visual Studio, CodeWarrior, Xcode, Kylix, and GCC / CLion toolchains.